Introduction to Object-Oriented Programming in C++

Corso di Programmazione - Prof. Marco Bertini
Università di Firenze

---

1. Perché lo Sviluppo Object-Oriented?

La programmazione orientata agli oggetti (OO) offre numerosi vantaggi rispetto ad altri paradigmi:

Vantaggi principali

- **Struttura software migliorata**: il codice diventa più facile da:
  - Comprendere
  - Mantenere nel tempo
  - Estendere con nuove funzionalità
- **Componenti riutilizzabili**: possibilità di creare moduli software che possono essere:
  - Migliorati senza riscriverli da zero
  - Resi più generali per utilizzi diversi
  - Condivisi tra progetti differenti

Questi vantaggi riducono i costi di sviluppo e manutenzione nel lungo termine.

---

2. Approccio Strutturato vs Object-Oriented

Programmazione strutturata

La **programmazione strutturata** (structured programming) si concentra principalmente sulla struttura procedurale:

- Focus sulle funzioni e sulle procedure

- La funzionalità relativa agli stessi dati è spesso **distribuita** in diverse parti del sistema
- I dati e le operazioni sono **separati**

Paradigma Object-Oriented

Il **paradigma OO** si concentra invece sulla strutturazione del sistema attorno ai dati:

- I dati sono **incapsulati** dentro oggetti
- Tutta la funzionalità relativa a quei dati è, in teoria, **contenuta dentro l'oggetto stesso**
- Dati e operazioni sono **unificati**

Strutturato	Object-Oriented
Focus su procedure e funzioni	Focus su dati e oggetti
Dati separati dalle operazioni	Dati incapsulati con operazioni
Funzionalità sparse nel sistema	Funzionalità localizzata negli oggetti

Table 1: Confronto tra approcci di programmazione

---

3. I Tre Concetti Chiave della Programmazione OO

L'idea principale della programmazione orientata agli oggetti è **incapsulare i dati dentro un "oggetto"**. Da questa idea derivano tre concetti fondamentali:

3.1 Data Abstraction (Astrazione dei Dati)

I dati sono visti in termini delle **operazioni che possiamo eseguire su di essi**, non solo come puri valori.

Componenti dell'astrazione

- **ADT (Abstract Data Types)**: specificano cosa si può fare con i dati

- **Information Hiding**: i dettagli del formato dei dati e dell'implementazione delle operazioni sono nascosti al codice che usa l'oggetto
- **Classi C++**: il meccanismo principale per implementare l'astrazione dei dati

Perché l'astrazione è importante

> "L'astrazione è un ottimo strumento ingegneristico: permette di specificare le proprietà esterne di un'entità senza dover specificare i dettagli interni. Possiamo così usare l'entità senza dover gestire i dettagli implementativi."

Beneficio pratico: progettare un sistema complesso diventa più semplice, perché possiamo gestire i diversi livelli di astrazione senza dover affrontare i dettagli più complessi dei livelli sottostanti.

Esempio pratico

Quando usi un oggetto `Stack`:

- Vedi le operazioni: `push()`, `pop()`, `top()`
- **Non vedi** se è implementato con un array, una lista, o altra struttura
- Puoi cambiare l'implementazione interna senza modificare il codice che usa lo stack

---

3.2 Inheritance (Ereditarietà)

L'**ereditarietà** permette di creare gerarchie di classi basate su relazioni di generalizzazione/specializzazione.

Come funziona

- Alcune classi di oggetti sono **generalizzazioni** di altre classi più specifiche
- La funzionalità generale viene messa in una **classe base** (base class)
- La funzionalità specifica viene messa in **classi derivate** (derived classes)
- Le classi derivate **ereditano** (o estendono) la funzionalità dalla classe base

Esempio concettuale

```
// Classe base generale
class Veicolo {
void accendi();
void spegni();
};

// Classe derivata specializzata
class Automobile : public Veicolo {
void cambiaMarcia(); // Funzionalità specifica
};

// Altra classe derivata
class Bicicletta : public Veicolo {
void pedalare(); // Funzionalità diversa
};
```

L'ereditarietà permette di riutilizzare il codice comune ed estenderlo per casi specifici.

3.3 Polymorphism (Polimorfismo)

Il **polimorfismo** permette di usare oggetti di tipi diversi nello stesso modo, e il sistema (C++, Java, Python) determina automaticamente quale funzionalità invocare in base al tipo effettivo dell'oggetto.

Forme di polimorfismo

- **Stesse operazioni su classi/oggetti diversi**: gli operatori aritmetici funzionano sia su interi che su numeri floating point, anche se l'implementazione è diversa
- **Virtual functions**: in C++, permettono di chiamare il metodo corretto in base al tipo effettivo dell'oggetto a runtime
- **Operator overloading**: ridefinizione di operatori standard per tipi definiti dall'utente

Esempio pratico

```
class Forma {
public:
virtual void disegna() = 0; // Metodo virtuale puro
};

class Cerchio : public Forma {
public:
void disegna() override { /* disegna cerchio */ }
};

class Quadrato : public Forma {
public:
void disegna() override { /* disegna quadrato */ }
};

// Uso polimorfico
Forma* forme[2];
forme[0] = new Cerchio();
forme[1] = new Quadrato();

for (int i = 0; i < 2; i++) {
forme[i]->disegna(); // Chiama il metodo corretto automaticamente!
}
```

Il polimorfismo rende il codice più flessibile ed estensibile.

---

4. Oggetti e Classi

4.1 Cos'è un Oggetto?

Un **oggetto** è come una variabile ordinaria:

- È un'istanza effettiva di un'entità software
- Contiene informazioni reali, possibilmente su qualcosa del mondo reale
- Mantiene informazioni
- È la realizzazione software di una "cosa" del mondo reale
- Può avere certe operazioni applicate su di esso

Esempio: un oggetto `studente` potrebbe contenere nome, matricola, voti e avere operazioni come `calcolaMedia()`, `aggiungiVoto()`.

4.2 Cos'è una Classe?

Una **classe** è (come) un tipo:

- È un'astrazione di un insieme di oggetti che si comportano identicamente
- Unisce l'implementazione di dati e operazioni
- Definisce l'implementazione interna e le operazioni
- Si possono creare (istanziare) oggetti da una classe, proprio come per tipi built-in (`int`, `float`)

Relazione tra classe e oggetto

- Una **classe** è la rappresentazione di un'idea, un concetto nel codice
- Un **oggetto** di una classe rappresenta un esempio particolare di quell'idea

Analogia:

- `int` è un tipo → la classe `Studente` è un tipo
- `int x = 5;` crea una variabile → `Studente s("Mario");` crea un oggetto

4.3 Perché le Classi Sono Importanti

Le classi aiutano a:

1. **Organizzare il codice**: struttura chiara e logica
2. **Ragionare sui programmi**: concetti espliciti nel codice
3. **Rendere esplicite le relazioni**: senza classi, un lettore deve indovinare le relazioni tra dati e funzioni; con le classi, queste relazioni sono esplicite e comprese dal compilatore
4. **Riflettere la struttura di alto livello**: più struttura del programma è riflessa nel codice stesso, non solo nei commenti

---

4.4 Visualizzazione: Classe vs Oggetti

Classe (concetto astratto):

Studente

- nome
- matricola
- voti[]

- calcolaMedia()
- aggiungiVoto()

Oggetti (istanze concrete):

Studente s1 Studente s2 Studente s3
nome: "Mario" nome: "Laura" nome: "Paolo"
matricola: 12345 matricola: 12346 matricola: 12347
voti: [28,30,27] voti: [25,29] voti: [30,30,28]

Tutti e tre gli oggetti hanno la stessa struttura (definita dalla classe), ma contengono dati diversi.

---

5. Programmazione Procedurale vs Object-Oriented

5.1 Visione Procedurale

Un programma può essere visto come una **sequenza di chiamate a procedure**:

- Il programma principale è responsabile di passare i dati alle singole chiamate
- I dati sono processati dalle procedure
- Le procedure sono separate dai dati

Struttura:

Main program (data)
↓
Procedure 1 → Procedure 2 → Procedure 3

5.2 Visione Object-Oriented

Un programma può essere visto come una **rete di oggetti interagenti**:

- Ogni oggetto gestisce il proprio stato interno
- Gli oggetti comunicano tra loro tramite messaggi (chiamate a metodi)
- I dati e le operazioni sono incapsulati insieme

Struttura:

Object 1 ← → Object 2
(data) (data)
↕ ↕
Object 3 ← → Object 4
(data) (data)

Procedurale	Object-Oriented
Sequenza di procedure	Rete di oggetti interagenti
Main controlla tutto	Ogni oggetto gestisce sé stesso
Dati passati alle funzioni	Dati incapsulati negli oggetti
Focus su algoritmi	Focus su struttura dati

Table 2: Confronto tra paradigmi

---

6. Oggetti Senza Classi

Non tutti i linguaggi OO usano classi nel modo tradizionale.

JavaScript e i Prototipi

JavaScript è un linguaggio orientato agli oggetti che **non ha classi** (almeno fino a ECMAScript 5):

- Usa **prototipi** (prototype-based o instance-based)
- L'ereditarietà delle classi diventa **riuso del comportamento** attraverso "decorazione" di oggetti esistenti che servono da prototipi

- ECMAScript 6 (ES6) ha introdotto le classi anche in JavaScript (come sintassi zuccherata sui prototipi)

Questo dimostra che la programmazione OO è un concetto più ampio delle classi stesse.

---

7. Metodi Object-Oriented

I **metodi OO** sono un insieme di tecniche per analizzare, decomporre e modularizzare le architetture dei sistemi software.

Principio fondamentale

- In generale, i sistemi evolvono e la funzionalità cambia
- Ma gli oggetti (e le classi di oggetti) tendono a **rimanere stabili nel tempo**
- Questo rende più facile mantenere ed evolvere il sistema

Esempio: in un sistema bancario, gli oggetti `ContoCorrente`, `Cliente`, `Transazione` rimangono stabili anche se cambia il modo in cui si calcolano gli interessi o si gestiscono i bonifici.

Fasi dello Sviluppo OO

Il paradigma OO influenza l'intero processo di sviluppo software, che procede attraverso fasi (anche se i confini sono spesso sfumati):

1. **OO Analysis** (Analisi OO)
2. **OO Design** (Progettazione OO)
3. **OO Programming** (Programmazione OO)

---

8. OO Analysis (Analisi Object-Oriented)

L'**Object-Oriented Analysis (OOA)** applica tecniche di modellazione a oggetti per analizzare i requisiti funzionali di un sistema.

Obiettivi dell'OOA

- Identificare la funzionalità richiesta
- Identificare classi e loro relazioni
- Guardare al **dominio del problema**
- Produrre un modello concettuale delle informazioni che esistono nell'area analizzata
- Comprendere i requisiti

Strumenti

Spesso usa strumenti del **Unified Modelling Language (UML)** come:

- **Class diagrams** (diagrammi delle classi)
- **Use cases** (casi d'uso)

Caratteristica importante

I modelli di analisi **non considerano vincoli implementativi** che potrebbero esistere. Si concentrano su **cosa** il sistema deve fare, non su **come** lo farà.

---

9. OO Design (Progettazione Object-Oriented)

L'**Object-Oriented Design (OOD)** elabora i modelli di analisi per produrre specifiche implementative.

Processo dell'OOD

Durante l'OOD, gli sviluppatori:

1. Creano astrazioni e meccanismi necessari per soddisfare i requisiti comportamentali determinati durante l'OOA
2. Mappano i concetti del modello di analisi su classi implementative
3. Producono un modello del **dominio della soluzione**
4. Creano una descrizione dettagliata di come il sistema sarà costruito

OOA vs OOD

OOA	OOD
Cosa il sistema fa	Come il sistema lo fa
Dominio del problema	Dominio della soluzione
Indipendente dall'implementazione	Considera vincoli implementativi
Modello concettuale	Modello implementativo

Table 3: OOA vs OOD

Indipendenza dal linguaggio

L'OOD è **relativamente indipendente** dal linguaggio di programmazione usato. Gli stessi concetti di design possono essere implementati in C++, Java, Python, ecc.

---

10. Obiettivi del Design

Un buon design software dovrebbe creare software che è:

Obiettivi primari

- **Facile da cambiare ed estendere**: flessibilità
- **Altamente coeso**: ogni modulo ha una responsabilità chiara e focalizzata
- **Debolmente accoppiato**: le dipendenze tra moduli sono minimizzate

Attività di design

1. Decomporre il sistema in moduli
2. Determinare le relazioni tra moduli
3. Identificare dipendenze e forme di comunicazione
4. Determinare le classi richieste
5. Definire uso, ereditarietà e composizione delle classi

Principio chiave: **alta coesione, basso accoppiamento** (high cohesion, low coupling).

---

11. OO Programming (Programmazione Object-Oriented)

La **Object-Oriented Programming (OOP)** si occupa principalmente di questioni di linguaggio di programmazione e implementazione software.

Definizione

OOP è un paradigma di programmazione che usa "oggetti" – strutture dati composte da dati e metodi insieme alle loro interazioni – per progettare applicazioni e programmi.

Visione di un programma OO

Un programma object-oriented può essere visto come una **collezione di oggetti interagenti**, in contrasto con il modello strutturale dove un programma è visto come una lista di task (subroutine) da eseguire.

Caratteristiche

- Implementa i concetti di OOD in un linguaggio specifico
- Focus su codifica, test e integrazione
- Utilizza le feature OO del linguaggio: classi, ereditarietà, polimorfismo

---

12. C++ e la Programmazione Object-Oriented

La citazione di Alan Kay

> "I invented the term 'Object-Oriented', and I can tell you I did not have C++ in mind."
> — Alan Kay

Questa citazione evidenzia che C++ non è un linguaggio OO "puro" come Smalltalk (il linguaggio che Kay stava sviluppando), ma piuttosto un linguaggio **multi-paradigma**.

---

13. Storia di C++

Origini

- **Inventore**: Bjarne Stroustrup, inizio anni '80
- **Caratteristica**: non è solo un linguaggio OOP, è **multi-paradigma**
- **Prima release commerciale**: 1985, era un preprocessore per C

Standard

- **ISO Standard**: 1998 (C++98), aggiornato nel 2003
- **Nuovi standard**: C11, C14, C17, C20, C++23
- Importante controllare quale standard supporta il proprio compilatore

---

14. C++ e C: Relazione

C++ è un **discendente diretto di C** che:

- Mantiene quasi tutto C come sottoinsieme
- Fornisce type checking più rigoroso rispetto a C
- Supporta direttamente una gamma più ampia di stili di programmazione rispetto a C

Nota: non tutto il codice C è valido C++, ma la maggior parte lo è.

---

15. C++ come Federazione di Linguaggi

C++ può essere considerato come una **federazione di linguaggi**; i principali "sub-linguaggi" sono:

15.1 C

C++ supporta ogni tecnica di programmazione supportata da C, inclusa la programmazione procedurale.

```
int somma(int a, int b) {
return a + b; // Stile C puro
}
```

15.2 Object-Oriented C++

Con classi, incapsulamento, ereditarietà, polimorfismo, ecc.

```
class Punto {
private:
int x, y;
public:
Punto(int x, int y) : x(x), y(y) {}
void muovi(int dx, int dy);
};
```

15.3 Generic Programming C++

Usa i template per creare codice generico riutilizzabile.

```
template<typename T>
T max(T a, T b) {
return (a > b) ? a : b;
}
```

15.4 Functional Programming

C++11 ha introdotto le **lambda** (funzioni anonime).

```
auto somma = [](int a, int b) { return a + b; };
cout << somma(3, 4) << endl; // Output: 7
```

Questa varietà rende C++ estremamente potente ma anche complesso da padroneggiare completamente.

---

16. Il Primo Programma C++: "Hello World"

Codice sorgente

```
#include <iostream>

int main() {
// let's greet
std::cout << "Hello, ";
std::string name = "world";
```

```
std::cout << name << std::endl;
}
```

Compilazione

```
g++ -Wall -o hello hello.cpp
```

Spiegazione

- `#include <iostream>`: include la libreria di I/O standard
- `std::cout`: oggetto per l'output su console (namespace `std`)
- `<<`: operatore di inserimento (overloaded)
- `std::endl`: inserisce newline e svuota il buffer
- `std::string`: classe per gestire stringhe (meglio di `char*`)
- `-Wall`: abilita tutti i warning del compilatore (buona pratica)

---

17. Gestione di Progetti: Makefiles

Problema

Progetti reali (real world) richiedono la gestione di:

- Multipli file sorgente
- Dipendenze tra file
- Librerie esterne
- Ordine di compilazione

Soluzione: Makefile

Un **makefile** è un file di testo che descrive:

- Come costruire i target (eseguibili e librerie)
- Dipendenze a livello di sorgente
- Dipendenze nell'ordine di build

Il comando `make` legge il makefile ed esegue le operazioni necessarie.

IDE e Makefiles

Gli IDE moderni gestiscono automaticamente i makefile:

- **Eclipse**: gestisce makefiles internamente
- **CLion**: usa **CMake** per generare makefiles

---

18. CMake: Generazione di Makefiles

CMake è un sistema che:

- Genera makefiles (o progetti IDE) da una descrizione di alto livello
- È multipiattaforma (Windows, Linux, macOS)
- Usa file `CMakeLists.txt` per descrivere il progetto

Workflow

CMakeLists.txt → CMake → Makefile → Make → Eseguibile

CLion è un IDE basato su CMake, che semplifica molto il processo.

---

19. Materiale di Lettura Consigliato

Libri di testo

- M. Bertini, "Programmazione Object-Oriented in C++" - cap. 1
- L.J. Aguilar, "Fondamenti di programmazione in C++. Algoritmi, strutture dati e oggetti" - cap. 1
- D.S. Malik, "Programmazione in C++" - cap. 1
- Thinking in C++, 2nd ed., Volume 1, cap. 1 pp. 23-30

Conclusioni

Questo documento ha introdotto i concetti fondamentali della programmazione object-oriented e del linguaggio C++:

- I tre pilastri della programmazione OO: astrazione, ereditarietà, polimorfismo
- La differenza tra oggetti e classi

- Il confronto tra paradigma procedurale e OO
- Le fasi dello sviluppo OO: analisi, design, programmazione
- C++ come linguaggio multi-paradigma
- Gli strumenti per lo sviluppo: compilatori, makefiles, CMake

Questi concetti costituiscono le fondamenta per comprendere la programmazione object-oriented in C++ e preparano per argomenti più avanzati come classi, ereditarietà, polimorfismo e template.

Crediti

Queste slide sono basate (pesantemente) sul materiale di:

- Dr. Ian Richards, CSC2402, University of Southern Queensland
- Dr. Douglas C. Schmidt, Washington University, St. Louis
- Prof. Marco Bertini, Università di Firenze